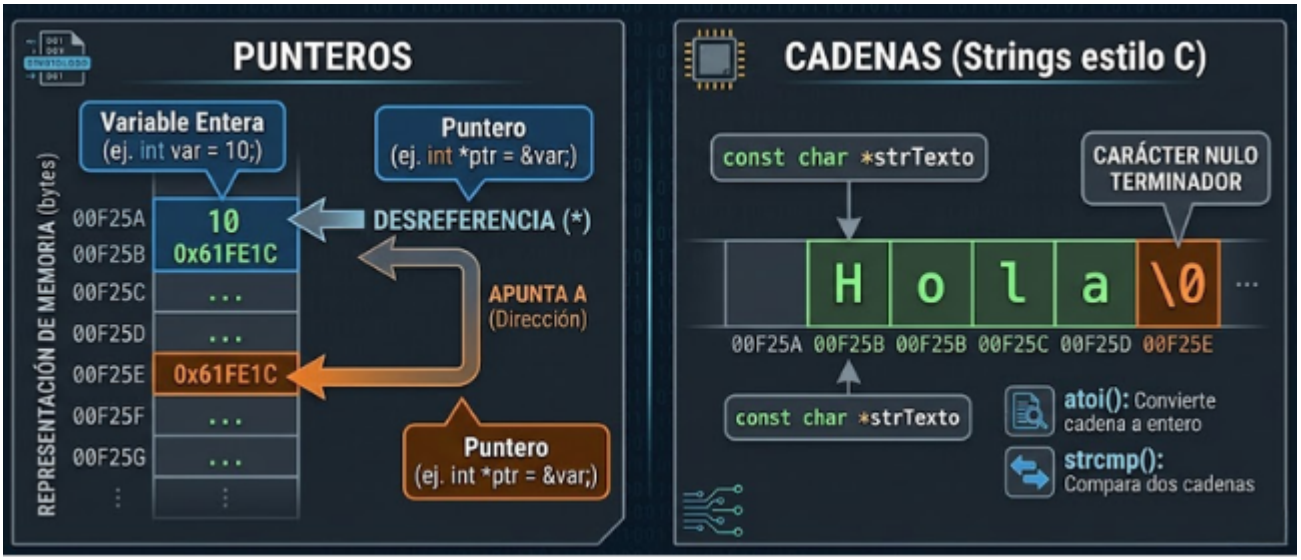


1° CUATRIMESTRE

📝📺📂 ACTIVIDAD 4: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++ - LPR 5° 3° A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

🎯 OBJETIVOS DE LA ACTIVIDAD

Para comprender cómo funciona la programación de bajo nivel, es necesario analizar qué ocurre en la memoria RAM de la computadora. La RAM se organiza como casilleros numerados con direcciones físicas únicas expresadas en formato hexadecimal.

El objetivo: Desarrollar un sistema básico de control de acceso por edad en C++ manipulando los datos en la memoria RAM mediante el uso directo de **punteros** y **desreferenciación**.

🎯 1. ¿Qué es un Puntero y la Desreferenciación?

Un **puntero** es una variable especial que almacena la dirección de memoria física (ej: 0x61fe1c) donde está alojada otra variable. **Desreferenciar** significa utilizar el operador asterisco (*) para acceder o modificar directamente el valor contenido dentro del casillero de memoria al que apunta el puntero.

📕 GLOSARIO (Conceptos técnicos que DEBEN dominar)

Como futuros Técnicos en Programación, el vocabulario técnico es su herramienta de presentación profesional:

- **Memoria RAM:** Espacio físico de almacenamiento temporal donde residen las variables durante la ejecución del programa.
- **Dirección Hexadecimal:** Representación numérica en base 16 (que inicia con 0x) para identificar un casillero exacto de la RAM.
- **Puntero (Pointer):** Variable que guarda la dirección de memoria de otra variable del sistema.
- **Operador de Dirección (&):** Operador que devuelve la dirección física en la RAM donde está guardada una variable.
- **Desreferenciar (*):** Operación para leer o modificar el valor contenido en la dirección de memoria apuntada.

🗂️ ESTRUCTURA DE CARPETAS DEL PROYECTO

Para mantener el estándar de la actividad y el control de versiones con Git, deben organizar la carpeta raíz llamada **CADENASPUNTEROS** exactamente de la siguiente manera:

```
cadenaspunteros/
├── .gitignore                  <-- Archivo para que Git ignore carpetas de configuración (.vscode)
├── LICENSE                     <-- Archivo de licencia MIT del proyecto (Uso escolar)
├── README.md                   <-- Archivo explicativo del uso de punteros en la RAM
├── docs/                       <-- Carpeta exclusiva para documentación
│   └── InformeProyectoCADENASPUNTEROS.pdf <-- ENTREGABLE (Informe PDF en Normas APA v7)
├── src/                        <-- Carpeta para los códigos fuente del proyecto
│   └── main.cpp                <-- Código fuente modular en C++
└── capturas/                   <-- Carpeta para guardar las evidencias locales (Imágenes .png)
    ├── ejecucion_punteros.png  <-- Captura de la terminal mostrando direcciones hexadecimales
    └── esquema_ram.png         <-- Diagrama de memoria hecho a mano en la carpeta
```

💻 EL RETO PRÁCTICO: CONTROL DE ACCESO CON PUNTEROS

Creen el archivo main.cpp dentro de la carpeta src/ con el siguiente código base provisto. Deben analizarlo, compilarlo y probar su funcionamiento en consola:

```
/*
  ================================================================
  Materia: Laboratorio de Programación (LPR)
  E.E.S.T. N° 1 "Eduardo Ader" — Vicente López
  Curso: 5° Año 3° División
  Profesor: Prof. York
  Archivo: main.cpp
  Objetivo: Control de accesos operando directamente sobre la RAM con punteros.
  ================================================================
*/
```

```cpp
#include <iostream>

using namespace std;

int main() {
    int edad = 0;
    int* p = &edad; // El puntero 'p' guarda la dirección de memoria física de 'edad'

    // ========================================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modifiquen la salida agregando su Nombre y Apellido reales.
    // ========================================================================
    cout << "======================================================" << endl;
    cout << "   CONTROL DE ACCESO RAM - ESTUDIANTE: Fuseneco Dardo " << endl;
    cout << "======================================================" << endl;

    cout << "=> Ingrese su edad: ";

    // ========================================================================
    // 1. CAPTURA MEDIANTE DESREFERENCIACIÓN (CÓDIGO EXPLICADO)
    // 💡 EXPLICACIÓN TÉCNICA:
    //    Usamos 'cin >> *p;' porque el asterisco '*' desreferencia al puntero.
    //    Esto le indica al compilador que debe guardar el número ingresado
    //    directamente en el casillero de memoria original de 'edad'.
    //    Si pusiéramos 'cin >> p;' sin asterisco, intentaríamos sobrescribir
    //    la dirección hexadecimal física de la RAM, provocando un error grave.
    // ========================================================================

    cin >> *p; // Captura el dato guardándolo directo en la memoria apuntada


    cout << "\n--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---" << endl;

    // ========================================================================
    // 2. CONDICIONAL CON PUNTERO DESREFERENCIADO
    // 💡 EXPLICACIÓN TÉCNICA:
    //    Evaluamos el valor alojado en la memoria usando '*p >= 18'.
    //    Si mostramos solo 'p', la consola imprimirá la dirección hexadecimal (ej: 0x61fe1c).
    // ========================================================================
    if (*p >= 18) {
        cout << "[ACCESO APROBADO] El usuario es mayor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    } else {
        cout << "[ACCESO RECHAZADO] Menor de edad." << endl;
        cout << "Edad registrada: " << *p << " anos." << endl;
        cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
    }

    cout << "======================================================" << endl;
    return 0; // Finalización exitosa de Windows (Código 0)
}
```

🏃 Comandos de PowerShell para compilar y correr en Windows 10/11 dentro de Visual Studio Code:

Abran la terminal integrada de VS Code (Ctrl + Ñ) e ingresen a la raíz de la carpeta cadenaspunteros.

1. Compilen el programa generando un ejecutable personalizado:
   g++ src/main.cpp -o src/cadenaspunteros.exe

2. Corran la aplicación generada en Windows:
   .\src\cadenaspunteros.exe

📋 CUESTIONARIO TÉCNICO DE CONTROL

Respondan las siguientes 3 preguntas dentro del informe PDF final que entregarán en Moodle.

🔬 Pregunta 1: Diferencia entre & y *

- **Explicación:** El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- **Pregunta:** ¿Qué valor imprime en pantalla la instrucción cout << p; comparada con cout << *p;?

🔬 Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Intentar escribir cin >> p; sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- **Pregunta:** ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?

🔬 Pregunta 3: Formato Hexadecimal

- **Explicación:** Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) antecedidas por 0x (ej: 0x61fe1c).
- **Pregunta:** ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

🏆 CRITERIOS DE ENTREGA Y EVALUACIÓN

⚠️ Modalidad y Entorno Híbrido:

- **Formato de Entrega del informe APA v7 en PDF:** Puede ser grupal (P.I.A) o individual. Si es grupal, el informe APA debe incluir detallado el nombre de cada integrante y qué tarea exacta hizo cada uno. Archivo guardado en su carpeta local como Docs/InformeProyectoCADENASPUNTEROS.pdf.
- **Compilación (Local vs. Online):** El uso de VS Code y MSYS2 en la PC es exclusivo para quienes tengan notebook propia con permisos de administrador. (por una limitación de accesos en algunos equipos de laboratorio). Si las PCs de la escuela les bloquea o no les dejan compilar, es obligatorio usar OnlineGDB para correr el código (hagan las capturas correspondientes).
- **El repositorio:** Sin importar si compilaste en tu computadora o en OnlineGDB, el repositorio propio de GitHub con toda la estructura de carpetas (src/, docs/, etc.) y el código subido con su modificación debe estar creado obligatoriamente. No se aceptan carpetas sueltas.
- **Carpeta de Campo analógica y digital del P.I.A. y Planilla de Proyectos:** Registro obligatorio de todas las actividades relacionadas exclusivamente con el P.I.A., no se deben dejar campos vacíos, se deben justificar con textos coherentes apuntando al P.I.A. (los campos son: **Objetivos para la semana; Entregas Parciales (APA + Código + Capturas); Obstáculos Encontrados y Acciones de Próxima Semana**).

📅 **Fecha de Entrega:** Prorrogada hasta el **17 de Septiembre de 2026**.

🛠️ Recursos Útiles (Links Oficiales)

- 🔗 Referencia Oficial C++ — Punteros y Memoria (cplusplus.com)
- 🔗 Documentación de Microsoft C++ — Operadores de Dirección y Desreferenciación
- 🔗 GeeksforGeeks — Pointers in C++